PROJECT OVERVIEW

BukuBesar — Core Banking

An event-driven mini core banking system: every transfer is a saga over Kafka, every rupiah lands in a double-entry ledger that always balances, and large transfers wait for a supervisor.

Raihan Evanza · Fullstack & Mobile Developer · portfolio-raihan-evanza.vercel.app

ROLE

Personal project · system design, backend, web, mobile, testing, demo deployment

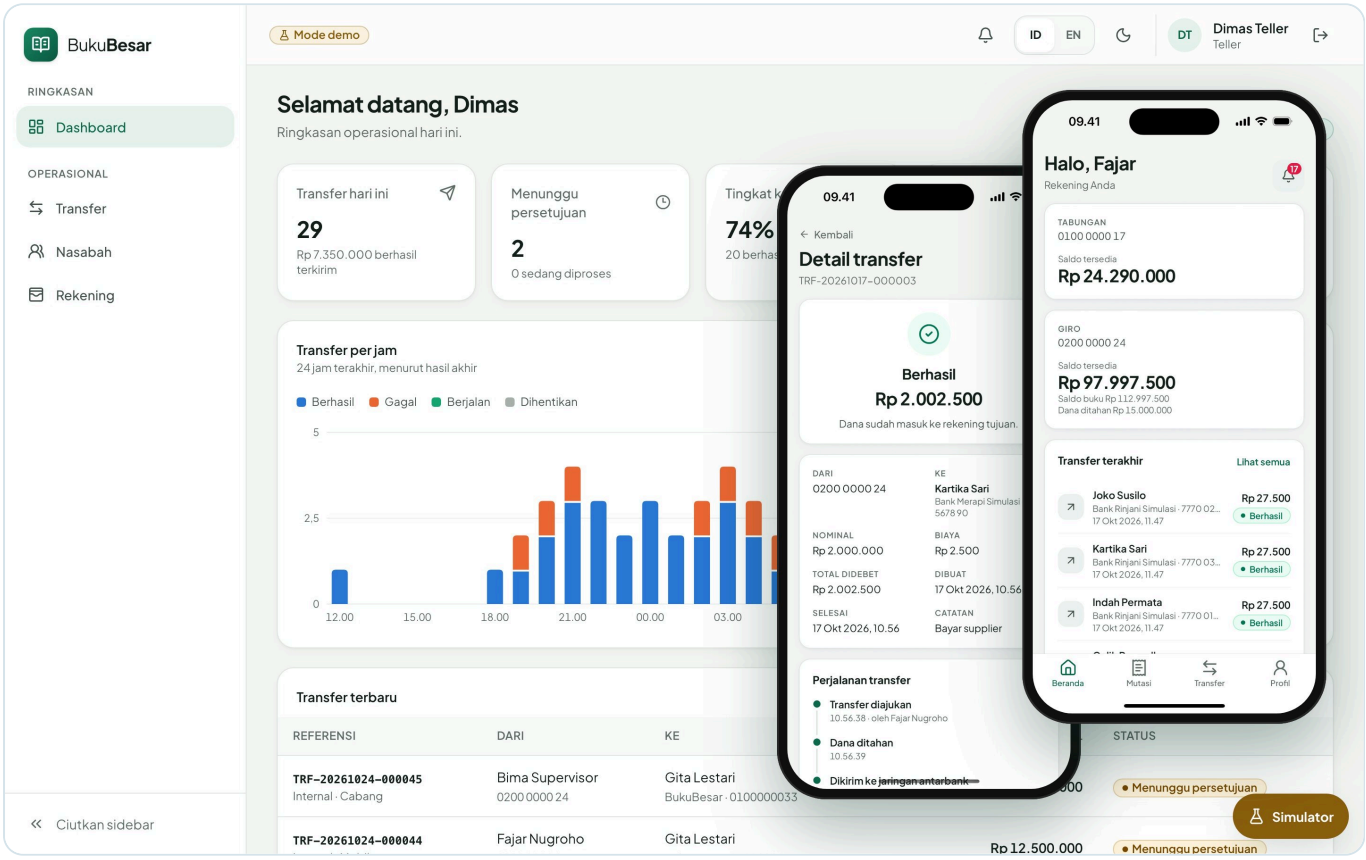
PLATFORMS

Web (staff back office) · Android & iOS (customer app) · three Spring Boot services

LINKS

Live demo: raihan-evanza-bukubesar.vercel.app/login?as=teller

Java 21 · Spring Boot 4.1 · Spring Kafka · PostgreSQL 16 · Flyway · Testcontainers · Vue 3.5 · Pinia · TanStack Query · React Native 0.87 · TypeScript · Tailwind CSS 4 · Rupa Design System · Docker



OVERVIEW

A single bank transfer touches many systems: limit checks, holding funds, debiting, crediting, the interbank network, notifications, and audit. If any step fails halfway, money must never disappear or be counted twice.

BukuBesar shows how to build that properly. Three Spring Boot services (accounts and ledger, transfers, notifications), each with its own database, talk over Kafka. Every transfer is a saga: funds are held, large transfers wait for a supervisor, then a double-entry journal is posted; when the interbank network rejects it or the destination is frozen, the funds are returned automatically (compensation).

The important claims are tested, not just stated: 50 concurrent transfers from one account never overdraw it, a service killed mid-saga resumes without losing anything, and replaying 1,700+ events changes nothing. A Vue 3 back

office, a React Native customer app, and a demo that runs the whole backend in the browser, all built on the Rupa design system (ledger brand).

KEY FEATURES

Transfer saga over Kafka

transfer-service orchestrates every transfer: HoldFunds → FundsHeld → (interbank network) → PostTransfer → TransferPosted → completed. Every failure after the hold releases it (compensation), including rejection, cancellation, and expired approvals.

Double-entry ledger

Every transfer becomes one journal: an interbank transfer has three lines (customer account, interbank settlement liability, fee income). The ledger is append-only, unbalanced journals are rejected by database triggers, and the trial balance always shows debits = credits.

Maker-checker

Transfers from Rp10,000,000 and account openings wait for a supervisor. The maker cannot approve their own request (err_self_approval), and two approvals at once give one 200 and one 409.

Event Inspector

Every Kafka message, dead letter topics included, is streamed to the web over Server-Sent Events with its topic, partition, offset, key, and JSON payload, so the saga can be watched as it runs.

Staff back office

A dashboard with KPIs, an hourly chart, and a live ledger check; an approvals inbox with countdowns; transfers with the saga timeline; customers and account opening; accounts with held funds, a running-balance statement, freeze, and cash deposits; the general ledger, trial balance, audit log, and limit settings.

Customer app

A React Native app (Android + iOS): accounts on the home screen, a filterable statement, internal and interbank transfers with a recipient check and confirmation, live status with the saga timeline, cancel while pending, and notifications.

Five roles, one permission map

Teller, Supervisor, Admin, Auditor, and Customer map to permissions in one file (Role.java). account-service signs RS256 JWTs and the other two services verify them through JWKS; both apps read the permission list from /api/auth/me to decide what to show.

Simulator, two languages, dark mode

In the demo, the Simulator panel plays the customer app with eight scenarios (internal, interbank, network rejection, frozen account, needs approval, insufficient funds, sent twice, 20 at once), switches staff persona, and resets the data. The interface is in Indonesian and English, light and dark, and works on a phone.

ARCHITECTURE & DATA FLOW

Web / app mobile
The Vue 3 back office and the React Native customer app call a single /api.

→

Edge (Caddy)
Routes each path to the service that owns it.

→

Tiga service Spring Boot
account (auth, accounts, ledger), transfer (saga, limits, approvals), notification (notifications, audit, inspector).

→

Kafka (Redpanda)
Saga commands and events via the outbox; three failures go to <topic>.DLT.

→

PostgreSQL 16
One database per service; no service reads another's tables.

- Events are not sent to Kafka directly: a service writes them to an outbox table in the same transaction as its data change, and a relay publishes them. Data and events never disagree.
- Every consumer records the events it has processed (processed_events), so redelivered messages are ignored. A message that fails three times (500 ms apart) or cannot be parsed goes to a dead letter topic and shows up in the Event Inspector.
- Saga state lives in the database and in Kafka, never in memory, so a service that dies mid-saga continues from where it stopped once it is back.

- In the Vercel demo, the three services, Kafka, and the databases are replaced by an in-browser backend with the same API contract and rules (a Kafka-like bus with partitions and offsets, SSE), keeping its data in localStorage.

STACK BY LAYER

Backend	Java 21 (virtual threads) · Spring Boot 4.1 · Spring Security (OAuth2 resource server, RS256 + JWKS) · Spring Data JPA · Flyway · springdoc-openapi · Maven multi-module
Messaging & data	Spring Kafka · Redpanda / Apache Kafka (Testcontainers) · PostgreSQL 16 (database per service) · Caddy 2 (edge)
Web	Vue 3.5 · TypeScript · Vite · Pinia · TanStack Query · VeeValidate + yup · vue-i18n · Tailwind CSS 4 · Rupa tokens (brand ledger) · Vercel
Mobile	React Native 0.87 (New Architecture, Hermes) · React Navigation 7 · Redux Toolkit + RTK Query · Keychain / Keystore · @rupa-ui/react-native
Testing	JUnit 5 + Mockito + AssertJ · Testcontainers (PostgreSQL, Kafka) · Vitest · Jest · e2e + chaos scripts (Node) · GitHub Actions

TECHNICAL DECISIONS

1. **A ledger that cannot be edited**
   Journal lines can only be added: database triggers reject UPDATE/DELETE and journals whose debits and credits differ. Corrections are new journals, as in real accounting, and every account balance equals the sum of its lines.

2. **Balances never go negative, even under races**
   Every hold is an UPDATE … WHERE available >= amount, backed by a balance ≥ 0 check constraint and unique constraints for one hold and one journal per transfer. Tested with 50 concurrent transfers from one account: exactly 33 succeed, 17 fail for insufficient funds, and the final balance is exact.

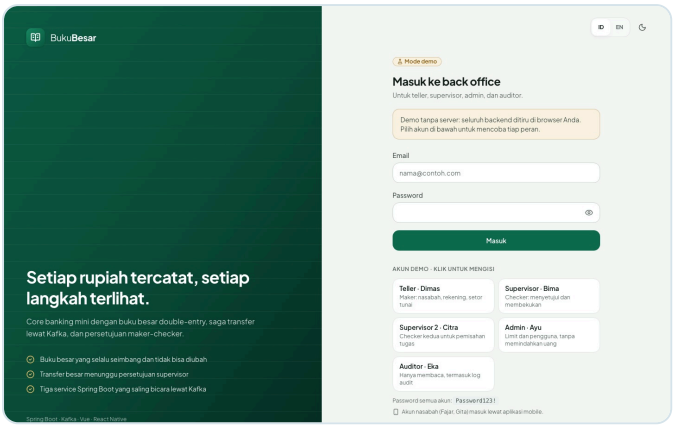
3. **Outbox + idempotent consumers**
   Kafka delivers at least once, so duplicates will happen. The outbox means no event is lost, and processed_events means none is applied twice. Rewinding every consumer group to the start (1,700+ messages) changes no data.

4. **Safe retries with an Idempotency-Key**
   Transfers and cash deposits require an Idempotency-Key stored with a request hash. The same key and body return the same transfer; the same key with a different body is refused with 422. The mobile app can retry on a flaky connection without risking a double transfer.

5. **Database per service and real Kafka in tests**
   No service reads another's tables; data crosses services only through events or APIs. Local development uses Redpanda to stay light on an 8 GB laptop, but an integration test runs the ledger flow on Apache Kafka through Testcontainers to prove the code is not Redpanda-specific.

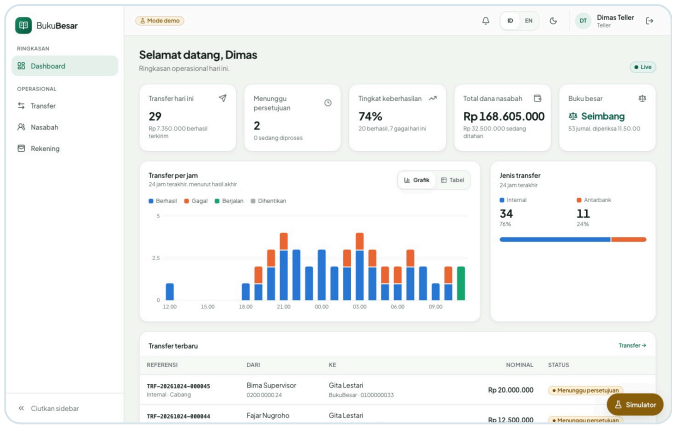
HOW TO USE IT

These steps follow the same order as the tutorial video.



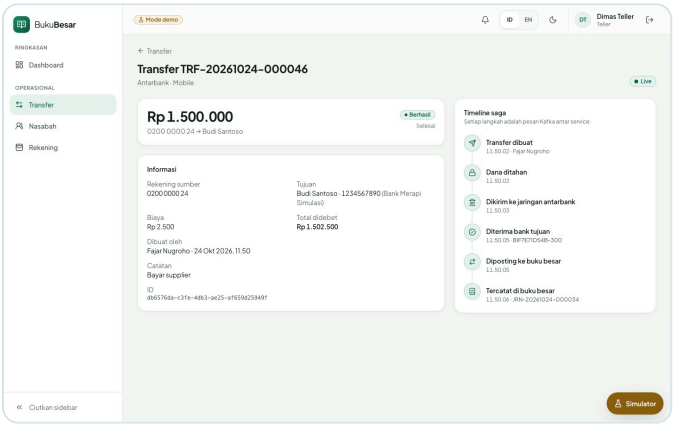
1 Sign in to the back office

Open the demo, click the Teller · Dimas card to fill in the email and password, then click Sign in. There are five staff accounts (Teller, Supervisor, Supervisor 2, Admin, Auditor), all with the password Password123!; customers use the mobile app. The /login?as=teller link signs you in as Dimas directly.



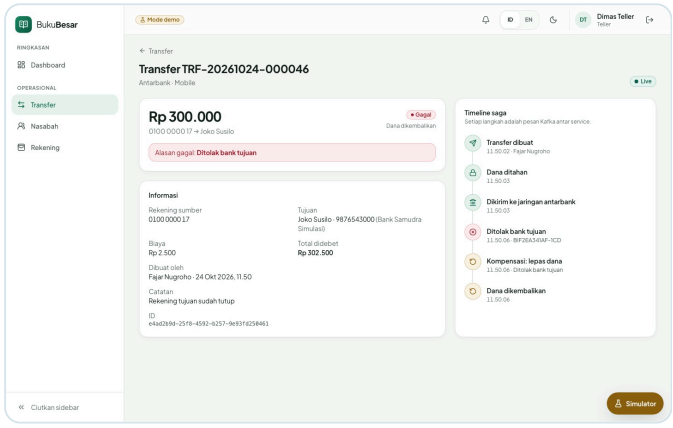
2 Read the dashboard

The cards at the top show transfers today, pending approvals, success rate, total customer deposits, and the ledger status (Balanced). Below them are hourly transfers by outcome, transfer types, and the latest transfers. Everything updates live without a refresh.



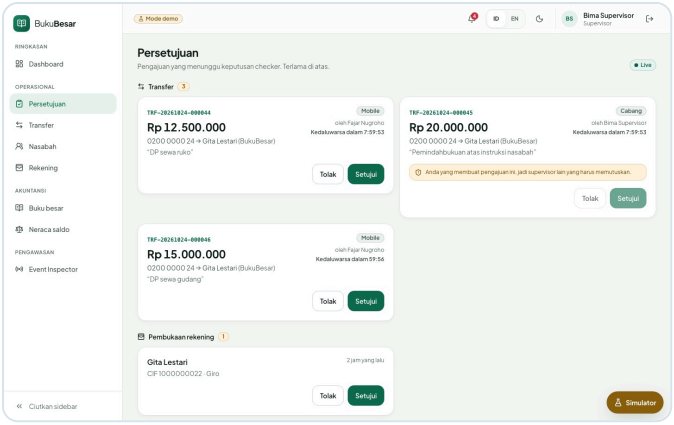
3 Send a transfer from the Simulator

Click Simulator at the bottom right. The panel plays the customer app: run Interbank Rp1,500,000, then open the transfer. The saga timeline shows every Kafka step: created, funds held, sent to the interbank network, accepted by the destination bank, posted, and recorded in the ledger.



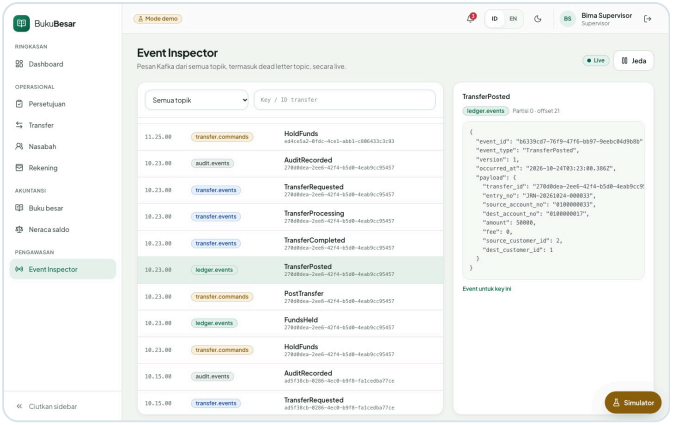
4 Watch a compensation

Run Interbank to a closed account. The interbank network rejects it, the transfer becomes Failed with the reason Rejected by the destination bank, and the saga compensates: the hold is released and the funds go back to the customer. To Hana's frozen account shows the same for a refused posting.



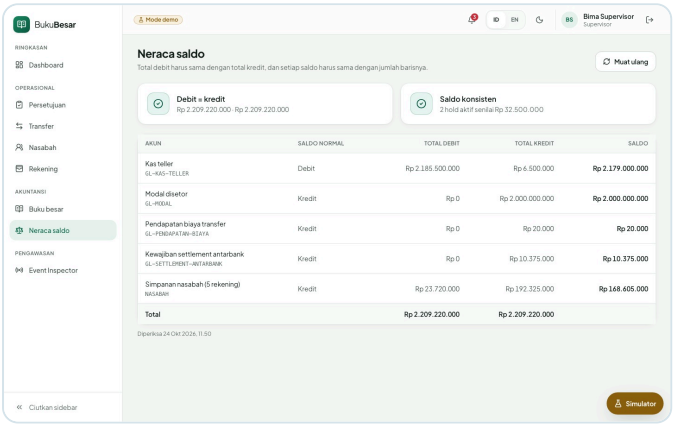
5 Approve as a supervisor

Run Transfer Rp15,000,000: the funds are held and the transfer waits for a supervisor. Switch persona to Supervisor (Bima) in the Simulator, then open Approvals. The Rp20,000,000 branch transfer Bima created is locked (maker-checker), so approve Fajar's transfer instead; the note is optional.



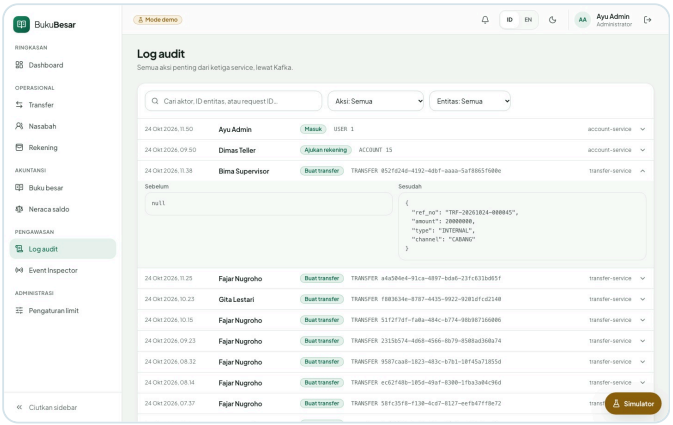
6 Follow the messages in the Event Inspector

Open the Event Inspector. Every Kafka message appears live with its topic (transfer.commands, ledger.events, audit.events, and the dead letter topics); click a row such as TransferPosted to see its partition, offset, and JSON payload. Filter by topic and search by key or transfer ID.



7 Check the ledger and trial balance

In the Simulator, run 20 transfers at once from Gita: some fail, the balance never goes negative. Then open Trial balance: total debits equal total credits and every balance matches the sum of its lines. Open General ledger and click a journal to see its debit and credit lines.



8 Read the audit log, switch theme and language

Switch persona to Admin (Ayu) and open Audit log: every action from all three services is recorded with before and after values. Use ID/EN and the moon icon in the header to switch language and theme. Reset demo data in the Simulator restores the initial data.

LIMITATIONS & NOTES

- The live demo runs without a server: the three services, Kafka, and the databases are emulated by a TypeScript backend in the browser that keeps data in each visitor's localStorage. Same API contract and rules, but no real JVM or Kafka.
- The real backend (three JVMs, Redpanda, PostgreSQL) runs only locally with docker compose; there is no hosted backend. The concurrency, compensation, and replay claims are tested against that local stack (36 backend tests, 20 end-to-end checks, chaos scripts).
- The interbank network is a simulator, not a real BI-FAST or switching integration.
- The customer app needs a running local backend and has not been published to the Play Store/App Store; its native projects still carry boilerplate names.

WHAT'S NEXT

- Cash withdrawals with approval, a transaction PIN in the mobile app, and push notifications.
- A dead letter viewer with replay, end-of-day interest (Spring Batch), and OpenTelemetry tracing.